

```

# a2a/agent_hierarchy.py
import uuid
from typing import Dict, List, Optional
from dataclasses import dataclass, field

@dataclass
class AgentIdentity:
    """Hierarchical agent identity system"""
    agent_id: str # Full hierarchical ID
    base_agent_id: str # Root agent ID without sub-agent tags
    parent_agent_id: Optional[str]
    agent_type: str # primary, sba (sub-agent), supervisor
    generation: int # 0 for root, 1 for first-level sub-agents, etc.
    created: str
    capabilities: List[str]

    def is_sub_agent(self) -> bool:
        return self.agent_type == "sba"

    def get_root_agent_id(self) -> str:
        """Get the root agent ID (remove all .sba. segments)"""
        parts = self.agent_id.split('.')
        root_parts = [part for part in parts if part != 'sba']
        return '.'.join(root_parts[:2]) # Return format: type_id

class AgentHierarchy:
    """Manages hierarchical agent relationships"""

    def __init__(self):
        self.agents: Dict[str, AgentIdentity] = {}
        self.children: Dict[str, List[str]] = {} # parent_agent_id -> list of child_agent_ids

    def generate_agent_id(self, base_name: str, agent_type: str,
                        parent_agent_id: Optional[str] = None) -> str:
        """Generate hierarchical agent ID"""

        if parent_agent_id:
            # Sub-agent generation
            if not parent_agent_id.endswith('.sba'):
                # First-level sub-agent
                return f"{parent_agent_id}.sba.{base_name}"
            else:
                # Nested sub-agent
                return f"{parent_agent_id}.{base_name}"

```

```

else:
    # Primary agent
    unique_id = str(uuid.uuid4())[:8]
    return f"{base_name}_{unique_id}"

def register_agent(self, base_name: str, agent_type: str,
                  parent_agent_id: Optional[str] = None,
                  capabilities: List[str] = None) -> AgentIdentity:
    """Register a new agent in the hierarchy"""

    agent_id = self.generate_agent_id(base_name, agent_type, parent_agent_id)

    # Calculate generation
    if parent_agent_id:
        parent_agent = self.agents.get(parent_agent_id)
        generation = parent_agent.generation + 1 if parent_agent else 1
    else:
        generation = 0

    agent_identity = AgentIdentity(
        agent_id=agent_id,
        base_agent_id=base_name,
        parent_agent_id=parent_agent_id,
        agent_type=agent_type,
        generation=generation,
        created=datetime.utcnow().isoformat(),
        capabilities=capabilities or []
    )

    self.agents[agent_id] = agent_identity

    # Update parent-child relationships
    if parent_agent_id:
        if parent_agent_id not in self.children:
            self.children[parent_agent_id] = []
        self.children[parent_agent_id].append(agent_id)

    return agent_identity

def get_agent(self, agent_id: str) -> Optional[AgentIdentity]:
    """Get agent identity by ID"""
    return self.agents.get(agent_id)

def get_sub_agents(self, parent_agent_id: str) -> List[AgentIdentity]:

```

```

        """Get all sub-agents of a parent agent"""
        child_ids = self.children.get(parent_agent_id, [])
        return [self.agents[child_id] for child_id in child_ids if child_id in self.agents]

def get_agent_lineage(self, agent_id: str) -> List[AgentIdentity]:
    """Get the complete lineage from root to this agent"""
    lineage = []
    current_agent_id = agent_id

    while current_agent_id in self.agents:
        agent = self.agents[current_agent_id]
        lineage.append(agent)
        if agent.parent_agent_id:
            current_agent_id = agent.parent_agent_id
        else:
            break

    return list(reversed(lineage)) # Root first

def is_descendant_of(self, agent_id: str, potential_ancestor_id: str) -> bool:
    """Check if an agent is a descendant of another"""
    lineage = self.get_agent_lineage(agent_id)
    ancestor_ids = [agent.agent_id for agent in lineage]
    return potential_ancestor_id in ancestor_ids

def get_capability_inheritance(self, agent_id: str) -> List[str]:
    """Get all capabilities inherited from parent agents"""
    lineage = self.get_agent_lineage(agent_id)
    all_capabilities = []

    for agent in lineage:
        all_capabilities.extend(agent.capabilities)

    return list(set(all_capabilities)) # Remove duplicates

# Global agent hierarchy
agent_hierarchy = AgentHierarchy()

```